

Staccato Language Design

Version 0.3 (Pre-Implementation)

Contents

1. Purpose	3
2. Design goals	3
3. Non-goals for version 1	3
4. Source structure	3
5. Timing model	4
6. Tempo and durations	4
7. Pitches and rests	5
8. Key signatures	5
9. Inline events	5
10. Named chords	6
11. Motifs	6
12. Phrases	6
13. Choosing the right construct	7
14. Tracks and instruments	9
15. Pitch conversion and validation	9
16. Intermediate representation	10
17. Compilation pipeline	10
18. Grammar sketch	11
19. Complete example: <i>Twinkle, Twinkle, Little Star</i>	12
20. Possible later additions	13

Version: 0.3 (pre-implementation)
File extension: .stc
Implementation language: C++
Compilation target: ChuckK

1. Purpose

Staccato is a compact, text-based music-sequencing language that compiles to ChuckK. It is intended for writing short melodic and harmonic arrangements with notes, rests, chords, reusable rhythms, reusable passages, and concurrent instrument tracks.

Staccato is not intended to represent every detail of conventional sheet music. Version 1 deliberately omits engraving, notation layout, and advanced performance markings. Its job is to make small musical programs concise, readable, statically checkable, and audible.

2. Design goals

- **Compact notation:** Common musical events should require little syntax.
- **Predictable time:** Events in one track execute from left to right and advance that track's time cursor automatically.
- **Simple polyphony:** Tracks execute concurrently; chords are simultaneous pitches within one track event.
- **Useful reuse:** Motifs reuse rhythm, chords reuse voicings, and phrases reuse passages.
- **Early diagnostics:** Invalid names, mismatched motif lengths, recursive phrases, and out-of-range pitches are reported before ChuckK code is emitted.
- **A small implementation:** A hand-written lexer and recursive-descent parser should be sufficient. No parser generator or compiler framework is required.

3. Non-goals for version 1

Version 1 does not include:

- Sheet-music engraving
- Measures with enforced time signatures
- Ties, tuplets, grace notes, or swing interpretation
- Dynamics, articulation, or per-note velocity
- Tempo changes within a piece
- Relative octaves
- User-defined instruments or synthesizer graphs
- General-purpose variables, loops, functions, or conditionals
- MIDI output

Optional barlines are accepted for readability, but have no timing semantics in version 1.

4. Source structure

A source file may contain, in any order:

- One **tempo** declaration
- Zero or one **key** declaration
- Chord definitions
- Motif definitions
- Phrase definitions
- Track definitions

Declarations may refer to definitions that occur later in the file. At least one track is required.

```
tempo 100
key { f# c# }

chord D = [d4 f#4 a4]
motif Pulse = { qn qn hn }

phrase Opening = {
  Pulse { d4 e4 f#4 }
}

track melody using flute {
  play Opening repeat 2
}
```

Braces delimit blocks. Whitespace and newlines otherwise have no syntactic meaning. `//` begins a comment that continues to the end of the line.

Identifiers may use letters, digits, and underscores, but may not begin with a digit. Capitalization is conventional, not semantic. The parser never decides what a name means based on its capitalization.

Chord, motif, and phrase names share one namespace. Track names occupy a separate namespace. Duplicate names are errors.

5. Timing model

Each track has an independent time cursor initially set to zero. Reading an event advances the cursor by that event's duration. Tracks begin simultaneously and execute concurrently.

A chord is one event containing multiple simultaneous pitches. It advances time once, by the chord's duration.

Tracks may have different total durations. The generated program remains alive until its longest track finishes. A shorter track ending before the longest one produces a warning, not an error.

6. Tempo and durations

Tempo is a positive number of quarter-note beats per minute:

```
tempo 120
```

Version 1 requires an integer tempo in the range 20 through 400 BPM.

Supported duration names are:

Name	Meaning	Quarter-note beats
wn	whole note	4
hn	half note	2
qn	quarter note	1
en	eighth note	1/2
sn	sixteenth note	1/4

Appending `.` dots a duration and multiplies it by 3/2. For example, **qn.** is one and a half quarter-note beats.

The compiler represents durations as exact rational values, not floating-point seconds. Conversion to ChuckK time occurs only during code generation.

7. Pitches and rests

A pitch consists of:

`note-name accidental? octave`

- Note names are lowercase **a** through **g**.
- **#** means sharp.
- **b** means flat.
- **!** means explicitly natural.
- The octave is an integer from **-1** through **9**, subject to MIDI bounds.

Examples:

```
c4      // middle C
f#5     // F-sharp 5
bb3     // B-flat 3
e!4     // explicitly natural E4
```

The spelling `bb3` is unambiguous: the first **b** is the note name and the second is the flat accidental.

A rest is written **r** and receives a duration exactly like a note:

```
r.qn
r.hn.
```

8. Key signatures

A key declaration lists the notes altered throughout the file:

```
key { f# c# }
```

Only note names plus **#** or **b** are permitted in the declaration; octaves are not written. With the declaration above, every **F** and **C** pitch without an explicit accidental is raised by one semitone.

An accidental written on a pitch always replaces the key signature alteration:

```
f4      // F-sharp because of the key signature
f!4     // F-natural
fb4     // F-flat
f#4     // explicitly F-sharp
```

Accidentals never accumulate. `f#4` under `key { f# }` is F-sharp, not F-double-sharp.

If no key declaration is present, all unmarked pitches are natural.

9. Inline events

Notes and rests use a dot followed by a duration:

```
c4.qn d4.qn e4.hn r.en g4.en
```

An inline chord encloses two or more pitches in brackets and has one duration:

```
[c4 e4 g4].hn
```

Every pitch in the chord begins and ends together. Rests are not permitted inside chords. A one-pitch bracket group is rejected because it is almost certainly accidental; the equivalent single-note syntax should be used.

The optional `|` token may appear between events as a visual separator:

```
c4.qn d4.qn e4.qn f4.qn | g4.wn
```

It does not reset time or validate a measure in version 1.

10. Named chords

A named chord stores an exact voicing:

```
chord C = [c4 e4 g4]
chord C_first_inversion = [e4 g4 c5]
```

The chord is used by attaching a duration to its name:

```
C.qn C_first_inversion.hn
```

Named and inline chords lower to the same intermediate event representation. Chord definitions contain at least two pitches and cannot contain rests.

11. Motifs

A motif stores a non-empty sequence of durations:

```
motif March = { qn qn hn }
```

Applying a motif supplies exactly one pitch value for every duration:

```
March { c4 d4 e4 }
```

A motif pitch value may be a note, a rest, a named chord, or an inline chord:

```
March { C r [d4 f#4 a4] }
```

The number of supplied values must equal the number of motif durations. A mismatch is a compile-time error pointing to both the motif definition and its application.

Motifs contain rhythm only. They cannot contain other motifs or phrases, and their expansion never depends on surrounding events.

12. Phrases

A phrase is a reusable sequence. It may contain inline events, motif applications, barlines, and `play` statements:

```
phrase Answer = {
  c4.qn e4.qn g4.hn
}
```

```
phrase CallAndAnswer = {
  g4.qn a4.qn b4.hn |
  play Answer
}
```

Phrase use is always introduced by `play`, eliminating ambiguity with named chord events:

```
play CallAndAnswer
play Answer repeat 3
play Answer transpose +5
play Answer repeat 2 transpose -12
```

`repeat` requires a positive integer. `transpose` takes a signed number of semitones. When both appear, the phrase is transposed and the resulting passage is repeated; for these two operations the observable result is independent of their order.

Phrase definitions may refer to definitions declared later. Direct and indirect recursive phrase references are compile-time errors.

Transposition affects every pitch in the expanded phrase, including pitches originating in named chords and motifs. It does not alter rests or durations.

13. Choosing the right construct

Inline notation is the default way to write music in Staccato. A user is never required to define a motif, chord, or phrase before writing a passage. Named constructs are conveniences for material that is repeated or is clearer when it has a musical name.

If the user wants to...	Use...
Play one note or rest	An inline event
Write a passage that will only occur once	A sequence of inline events
Play several pitches simultaneously once	An inline chord
Reuse the same chord voicing	A named chord
Reuse one rhythm with different pitch material	A motif
Reuse or transform a complete passage	A phrase
Play an independent instrument part concurrently	A track

13.1 One-off notes, rests, and passages

A short passage that is not reused should be written directly. It does not need a motif or phrase definition:

```
track melody using flute {
  c4.qn d4.en e4.en f4.hn |
  r.qn g4.qn a4.hn
}
```

Durations belong to their events, so a one-off passage may use any mixture of rhythms without first naming that rhythm.

13.2 One-off and reusable chords

Use brackets when a voicing occurs only once:

```
track harmony using rhodey {
  [c4 e4 g4].hn [d4 f4 a4].hn
}
```

Give the voicing a name when it appears repeatedly or when the name communicates its role more clearly:

```
chord C = [c4 e4 g4]
chord G = [g3 b3 d4]

track harmony using rhodey {
  C.hn G.hn C.wn
}
```

13.3 Reusing rhythm with a motif

A motif is useful when the same rhythm is applied to different pitch material:

```
motif ShortShortLong = { qn qn hn }
```

```

track melody using flute {
  ShortShortLong { c4 d4 e4 }
  ShortShortLong { g4 a4 b4 }
  ShortShortLong { C r [g4 b4 d5] }
}

```

If that rhythm occurs only once, inline notation is usually clearer:

```
c4.qn d4.qn e4.hn
```

Defining a motif for a single use is legal, but it is a stylistic choice rather than a language requirement.

13.4 Reusing a complete passage with a phrase

A phrase stores pitches and rhythms together. Use it when the complete passage will be repeated, nested, or transformed:

```

phrase Fanfare = {
  c4.qn e4.qn g4.hn
}

track brass using organ {
  play Fanfare
  play Fanfare transpose +5
  play Fanfare repeat 2 transpose +12
}

```

For a passage that occurs once and requires no transformation, writing the events directly in its track is preferable.

13.5 Mixing constructs

The constructs are designed to be mixed. A phrase may combine inline events, named chords, motif applications, rests, and other phrases:

```

tempo 112
key { f# }

chord D = [d4 f#4 a4]
chord G = [g3 b3 d4]
motif Rise = { en en qn hn }

phrase Theme = {
  Rise { d4 e4 f#4 a4 } |
  r.qn D.qn G.hn      |
  play Ending
}

phrase Ending = {
  e4.qn d4.qn d4.hn
}

phrase Backing = {
  D.hn G.hn |
  [a3 c#4 e4].wn |
  D.hn G.hn
}

```

```

track lead using flute {
    d4.qn e4.qn          // one-off inline events
    play Theme repeat 2  // reusable complete passage
}

track accompaniment using rhodey {
    r.hn
    play Backing repeat 2 // named voicings and an inline chord
}

```

Definitions by themselves produce no sound. Music is emitted only when an event or `play` statement is reached from a track body.

Version 1 intentionally has no anonymous duration-group syntax such as `qn { c4 d4 e4 }`. The equivalent one-off passage is written `c4.qn d4.qn e4.qn`. Duration groups may be considered later if real programs show that repeated suffixes harm readability.

14. Tracks and instruments

A track binds an event sequence to a built-in instrument:

```

track lead using flute {
    c5.qn d5.qn
    play Answer
}

```

Track bodies accept the same contents as phrase bodies. A file may contain multiple tracks, and each track name must be unique.

Version 1 supports a fixed, documented set of instrument names mapped to ChuckK STK classes. An initial implementation may support:

Staccato name	ChuckK class
flute	Flute
clarinet	Clarinet
bowed	Bowed
mandolin	Mandolin
sitar	Sitar
rhodey	Rhodey
organ	BeeThree
modalbar	ModalBar

Using an unknown instrument is a compile-time error. The table may be extended without changing the language grammar or IR.

15. Pitch conversion and validation

During lowering, pitches are converted to MIDI note numbers using $C4 = 60$.

The order of operations is:

1. Resolve the pitch's written accidental or the global key signature.
2. Convert the spelled pitch and octave to a MIDI number.
3. Expand named chords, motifs, and phrases.

4. Apply phrase transposition to every resulting pitch.
5. Validate the final MIDI value.

Values outside MIDI 0 through 127 are errors. Values outside the practical piano range 21 through 108 produce warnings.

ChuckK code generation converts MIDI values to frequencies. The source language does not expose frequencies or MIDI numbers directly.

16. Intermediate representation

All musical constructs lower to tracks containing flat event lists:

```
ProgramIR
  tempo_bpm
  tracks []

TrackIR
  name
  instrument
  events []

EventIR
  pitches[]           // zero, one, or many MIDI note numbers
  duration            // exact rational number of quarter-note beats
  source_origin       // original location plus expansion context
```

The pitch list determines the event kind:

- Empty list: rest
- One element: note
- Two or more elements: chord

Expansion context is retained so an error in a transposed phrase can mention both the original pitch and the play statement that caused the invalid value.

17. Compilation pipeline

17.1 Lexing

The lexer produces tokens with line and column information. Token kinds include keywords, identifiers, pitch literals, duration names, integers, punctuation, and end-of-file.

Pitch literals should be recognized as a single token. For example, `bb3` is one pitch token rather than an identifier followed by a number.

17.2 Parsing

A hand-written recursive-descent parser produces an AST that preserves source locations and unresolved names. Parsing does not expand motifs or phrases.

17.3 Name resolution

The compiler builds symbol tables, rejects duplicate or unknown names, verifies that a name is used as the correct kind of definition, and constructs a phrase dependency graph. A depth-first traversal rejects recursive phrase cycles.

17.4 Lowering and validation

The compiler applies key signatures, resolves chords, expands motifs and phrases, applies transformations, converts pitches to MIDI values, and emits the flat IR. It then performs pitch-range and track-duration checks.

17.5 Chuck generation

Each track becomes a Chuck function and is started as a concurrent shred. All track functions are sporked before the parent shred waits for the duration of the longest track.

A note uses one STK instrument voice. A chord uses a pool of independent STK instances so every pitch can sound simultaneously; repeated `noteOn` calls on a single monophonic STK instance are not assumed to form a chord. A rest only advances time.

At an event boundary, sounding voices receive `noteOff`; the next event then begins at the same logical time. Version 1 uses one fixed note amplitude.

18. Grammar sketch

This grammar is descriptive EBNF. Lexical details such as comments and token boundaries are handled by the lexer.

```
program          = declaration* EOF ;

declaration      = tempo_decl
                  | key_decl
                  | chord_decl
                  | motif_decl
                  | phrase_decl
                  | track_decl ;

tempo_decl       = "tempo" INTEGER ;
key_decl         = "key" "{" key_accidental* "}" ;
chord_decl       = "chord" IDENT "=" pitch_group ;
motif_decl       = "motif" IDENT "=" "{" duration+ "}" ;
phrase_decl      = "phrase" IDENT "=" sequence_block ;
track_decl       = "track" IDENT "using" IDENT sequence_block ;

sequence_block   = "{" sequence_item* "}" ;
sequence_item    = inline_event
                  | motif_application
                  | play_statement
                  | "|" ;

inline_event     = PITCH "." duration
                  | "r" "." duration
                  | IDENT "." duration
                  | pitch_group "." duration ;

motif_application = IDENT "{" motif_value+ "}" ;
motif_value      = PITCH | "r" | IDENT | pitch_group ;

play_statement   = "play" IDENT play_modifier* ;
play_modifier    = "repeat" INTEGER
```

```

| "transpose" SIGNED_INTEGER ;

pitch_group      = "[" PITCH PITCH+ "]" ;
duration         = ("wn" | "hn" | "qn" | "en" | "sn") "."? ;
key_accidental   = NOTE_NAME ("#" | "b") ;

```

The semantic checker, rather than the grammar, determines whether an IDENT is a chord, motif, phrase, or instrument name.

19. Complete example: *Twinkle, Twinkle, Little Star*

The following arrangement contains a melody and chord accompaniment. The two tracks begin together and have equal duration.

```

// twinkle.stc
tempo 100

chord C = [c4 e4 g4]
chord F = [f4 a4 c5]
chord G = [g3 b3 d4]

motif FourQuarters = { qn qn qn qn }
motif TwoAndHold = { qn qn hn }

phrase MelodyA = {
  FourQuarters { c4 c4 g4 g4 } |
  TwoAndHold { a4 a4 g4 } |
  FourQuarters { f4 f4 e4 e4 } |
  TwoAndHold { d4 d4 c4 } |
}

phrase MelodyB = {
  FourQuarters { g4 g4 f4 f4 } |
  TwoAndHold { e4 e4 d4 } |
}

phrase HarmonyA = {
  C.hn C.hn | F.hn C.hn |
  F.hn C.hn | G.hn C.hn |
}

phrase HarmonyB = {
  C.hn F.hn | C.hn G.hn |
}

track melody using flute {
  play MelodyA
  play MelodyB repeat 2
  play MelodyA
}

track harmony using rhodey {
  play HarmonyA
  play HarmonyB repeat 2
}

```

```
    play HarmonyA  
}
```

Conceptually, compilation produces two flat timelines:

```
melody:  C4/q C4/q G4/q G4/q A4/q A4/q G4/h ...  
harmony: C/h  C/h  F/h  C/h  F/h  C/h  G/h  C/h ...
```

The `melody` and `harmony` timelines run concurrently. Inside the harmony track, each named chord is one event, so all three chord pitches begin together and the track cursor advances by one half note.

20. Possible later additions

The following features can be added without invalidating the core event IR:

- Per-event velocity and dynamics
- Articulation and configurable note gate length
- Ties
- Named keys such as `key D major`
- Meter declarations and optional barline validation
- Tempo changes
- Built-in chord formulas such as `c4:maj7`
- Additional phrase transformations such as retrograde
- MIDI output
- Source-map comments in generated ChuckK

Tuplets and independently timed voices within one track may require a richer duration or timeline model and should be designed separately.